Replicating Figure 11.15: A Thread-Configuration Sweep of the Hybrid CPU+GPU Mandelbrot Renderer

Matias Saavedra

Friday 10th July, 2026

Binary: `0f782fd` (tag `binary-v0-buggy`) — original code, pre-bugfix.

Resumen

This report documents a follow-up profiling experiment on `mandelHybrid`, the hybrid CPU+GPU Mandelbrot renderer from Barlas, *Multicore and GPU Programming* (2nd ed., 2022), Section 11.5.1. The goal was to reproduce Figure 11.15 of that book—a sweep of end-to-end execution time across seven thread/GPU configurations—on a different machine (an i7-9750H + GTX 1660 Ti Max-Q laptop) and compare the resulting curve to the book's Ryzen 3900X + RTX 2060 Super result. A small instrumentation patch was added to the existing harness: an end-to-end `clock_gettime` wall-clock timer, a self-describing configuration banner, and two boolean flags (`profileQuiet`, `profileSave`) that suppress per-region `stderr` output and skip PNG writes when batch-running. A Bash driver (`sweep.sh`) executes seven configurations three times each into a CSV; a Python script (`plot_fig1115.py`) renders a chart in the style of Figure 11.15. The replication shows the same monotonic decrease as one moves from GPU-alone through hybrid configurations with more CPU workers, but the slope is much shallower than in the book: on this laptop the GPU is 2.1× faster than the CPU at full parallelism, so adding 11 CPU workers buys only a 30% speed-up over GPU-alone, versus the ∼50% the book observed when its CPU and GPU were roughly on par.

1. Background and Goals

Figure 11.15 of Barlas (2022) plots the average end-to-end execution time of a 100-frame, full-HD Mandelbrot animation as a function of thread configuration. Eight bars are shown, corresponding to: CPU-only with the full thread count of the machine, GPU-only, and six hybrid configurations with $1, 2, 4, 8, 16, 23$ pure CPU workers running alongside one GPU thread. A line overlay reports each bar's time as a percentage of the GPU-alone time, anchoring the comparison.

The book's measurements were taken on an AMD Ryzen 3900X (12 cores, 24 threads) paired with an NVIDIA RTX 2060 Super. On that platform the CPU and GPU are roughly on par for the Mandelbrot iteration kernel and the hybrid run cuts execution time approximately in half relative to either alone. This report addresses three questions:

1. What instrumentation is needed to record the metric the figure actually plots, given the existing per-region NVTX/CUDA-event profiling captures only components of total time and not the end-to-end wall clock?

2. How does the same workload behave on an Intel Core i7-9750H (6 cores, 12 threads) paired with an NVIDIA GeForce GTX 1660 Ti Max-Q—a laptop GPU that is roughly comparable in fp64 throughput to the RTX 2060 Super but mated to a CPU that is half as wide?

3. What is the shape of the curve and what does any deviation from the book's curve reveal about the CPU/GPU balance of this machine?

1.1. What Was Already Instrumented

A prior profiling pass had already added NVTX annotations and CUDA event timing to the program (documented in `01-initial-profile.tex`). Those mechanisms give:

- per-region GPU kernel time and device-to-host `memcpy` time, summed over the run and reported in `CUDAmemCleanup()`;
- per-region CPU compute time accumulated by a `thread_local` pair of counters in `mandelregion.cpp` and reported in `printCPUSummary()`;
- nested NVTX ranges (`examine region`, `CPU region compute`, `GPU region compute`) consumable by Nsight Systems.

What was not captured was the total end-to-end wall-clock time of the program—the makespan—which is exactly the quantity plotted in Figure 11.15. `cpuTotalMs` excludes idle time. `totalKernelMs` covers only the GPU's busy stretches. The thread summaries are independent and the final time is determined by whichever thread finishes last, plus any image-save tail. Two trivial routes existed: wrap the binary with `/usr/bin/time -f "%e"`, or bake a `clock_gettime` pair into `main.cpp`. The latter was chosen because it makes the binary self-reporting (easier for batch scripts to grep) and because it gives a clean point to also emit a configuration banner.

2. Instrumentation Changes

2.1. Two Global Flags Defined in main.cpp

Two booleans were added with `extern "C"` linkage so that the `nvcc`-compiled `kernel.o` and the `g++`-compiled `.o` files can share the symbol without name-mangling concerns:

Código 1: main.cpp — profiling flag definitions.

```
// Profiling flags consulted by kernel.cu, mandelregion.cpp and
// mandelframe.cpp.  Defined here so a single command-line switch
// controls them.  C linkage keeps the symbol name identical in nvcc
// and g++ object files.
extern "C" int profileQuiet = 0;   // 1 = suppress per-region stderr prints
extern "C" int profileSave  = 1;   // 0 = skip PNG saves (pure-compute timing)
```

Two new optional positional command-line arguments were added so the flags can be flipped without recompiling:

Código 2: main.cpp — argument parsing.

```
// Usage: spec_file [numThr] [GPUenable] [diffT] [pixT] [quiet] [save]
if (argc > 6) profileQuiet = atoi(argv[6]);
if (argc > 7) profileSave  = atoi(argv[7]);
```

2.2. Per-Region Print Gates

Three call sites had previously emitted one `stderr` line per leaf region. Across the 4,603 leaves of the standard run, plus the banners, an unmuted execution emits roughly 5,000 lines per run; 7 configurations × 3 reps would produce ∼100,000 lines of stderr noise that dwarfs the summary lines we actually want to parse. Each call site was wrapped in an `if (!profileQuiet)` check, with the per-thread `printCPUSummary()` and the `CUDAmemCleanup()` summary deliberately left always on:

Código 3: kernel.cu — GPU per-region print gated.

```
// Defined in main.cpp.  Suppresses per-region stderr lines when set,
// leaving the once-per-run summary intact.
extern "C" int profileQuiet;

// ... inside hostFE() ...
if (!profileQuiet)
  fprintf(stderr,
        "[GPU region %4ld] size %4dx%4d  kernel %.3f ms  D->H %.3f ms\n",
        totalRegions, resX, resY, kernelMs, memcpyMs);
```

Código 4: mandelregion.cpp — CPU per-region print gated.

```
extern "C" int profileQuiet;

// ... inside compute(), CPU path ...
if (!profileQuiet)
  fprintf(stderr,
        "[CPU region %4ld] size %4dx%4d  compute %.3f ms\n",
        cpuRegionCount, pixelsX, pixelsY, ms);
```

2.3. Skip-Save Gate

Figure 11.15 in the book includes the PNG-encoding tail; for side-by-side comparison the sweep also defaults to `save=1`, but the option to time pure compute is useful for follow-up analysis. The guard was added inside `MandelFrame::regionComplete`, the single point at which the atomically-decremented remaining-regions counter hits zero and the frame is written:

Código 5: mandelframe.cpp — PNG-save gate.

```
// Defined in main.cpp.  When 0, regionComplete skips img->save() so we
// can time the pure compute path without the PNG-encode tail.
extern "C" int profileSave;

void MandelFrame::regionComplete()
```

```
{
  if (remainingRegions.fetchAndAddOrdered(-1) == 1)  // last became 0
  {
    if (profileSave)
      img->save(fname, "PNG");
  }
}
```

2.4. End-to-End Wall-Clock Timer and Configuration Banner

The new timer brackets exactly the compute phase: it starts immediately before `CUDAmemSetup` (when GPU is enabled) and stops after the `wait()` on every worker thread returns. Image saves are naturally included because they occur on the worker thread that decrements `remainingRegions` to 0, before that worker exits its `run()` loop and is joined.

Código 6: main.cpp — banner, timer, machine-parseable output.

```
fprintf(stderr,
       "[config] numThr=%d gpuEnable=%d diffT=%.3f pixT=%d "
       "res=%dx%d frames=%d quiet=%d save=%d\n",
       numThreads, (int)enableGPU, diffT, pixT,
       resolutionX, resolutionY, numframes,
       profileQuiet, profileSave);

// ... thread launch ...

struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);

if (enableGPU) {
   CUDAmemSetup(resolutionX, resolutionY);
   thr[0]->run();
   CUDAmemCleanup();
} else {
   thr[0]->run();
}
for (int i = 1; i < numThreads; i++)
   thr[i]->wait();

clock_gettime(CLOCK_MONOTONIC, &t1);
double elapsed_s = (t1.tv_sec - t0.tv_sec)
             + (t1.tv_nsec - t0.tv_nsec) * 1e-9;

// Machine-parseable line: sweep.sh greps for this exact prefix.
fprintf(stderr, "[total_elapsed_s] %.6f\n", elapsed_s);
```

`CLOCK_MONOTONIC` is used for the same reason as in the per-region instrumentation: it is immune to wall-clock jumps from NTP or DST. The marker line `[total_elapsed_s]` is chosen because it is unique across all stderr output and is

therefore safe to extract with a single `grep ... \| awk '{print $2}'`.

2.5. Build-System Adjustment (nvcc + gcc 16)

A separate environmental issue surfaced during the rebuild: the host `g++` on the test machine had been upgraded to 16.1.1, but `nvcc` 13.2 only supports up to GCC 15. The compile failed with `type_traits` errors deep inside libstdc++. The Makefile was amended to pin the host compiler explicitly via `-ccbin`:

Código 7: Makefile — pin nvcc's host compiler.

```
# nvcc 13.2 does not support gcc 16 yet; pin to g++-15 for the host
# compiler and for nvcc's -ccbin so both halves of the program share
# an ABI.
HOSTCC   = g++-15
NVCC     = nvcc -ccbin $(HOSTCC)
CC       = $(HOSTCC)
```

Using the same compiler for both halves of the build is important because the C++ runtime types (e.g. `std::atomic_int`, `std::vector`) flow across the `nvcc`/`g++` boundary via the shared `MandelRegion`/`MandelFrame` headers. A divergent ABI here would manifest as silent corruption rather than a link error.

3. Batch Driver and Plot Script

3.1. sweep.sh

The driver script runs seven configurations N times each and emits a single CSV row per run. Configurations are tuned for a 12-thread machine: total threads (the `numThreads` argument to `mandelHybrid`) never exceed 12, and the count of pure CPU workers in a hybrid run is `numThreads - 1` because thread 0 is the GPU driver.

Código 8: sweep.sh — configuration matrix and main loop.

```
CONFIGS=(
  "CPU12      12  0"
  "GPU        1  1"
  "GPU+1CPU     2  1"
  "GPU+2CPU     3  1"
  "GPU+4CPU     5  1"
  "GPU+8CPU     9  1"
  "GPU+11CPU    12  1"
)

CSV="$OUT_ABS/results.csv"
echo "label,numThreads,gpuEnable,rep,elapsed_s" > "$CSV"

for entry in "${CONFIGS[@]}"; do
  set -- $entry
  label=$1; nthr=$2; gpu=$3
  for r in $(seq 1 "$REPS"); do
    rm -rf "$SCRATCH" && mkdir -p "$SCRATCH"
```

```
    log="$OUT_ABS/logs/${label}.r${r}.log"
    pushd "$SCRATCH" >/dev/null
    # quiet=1, save passed through (default 1 to match Fig 11.15).
    "$BIN_ABS" "$SPEC_ABS" "$nthr" "$gpu" "$DIFFT" "$PIXT" 1 "$SAVE" \
      > "$log.stdout" 2> "$log.stderr"
    popd >/dev/null
    elapsed=$(grep '^\[total_elapsed_s\]' "$log.stderr" | awk '{print $2}')
    echo "$label,$nthr,$gpu,$r,$elapsed" >> "$CSV"
  done
done
```

Two design choices are worth highlighting:

- Each run is executed with its own scratch `cwd`, so the PNG files for run r of config c do not pollute the next invocation and disk usage stays bounded (50 MB peak for a single full-HD 100-frame run, cleaned between runs).
- Configurations, reps count, save flag, and thresholds are all environment-variable overrides (`REPS`, `SAVE`, `DIFFT`, `PIXT`, `SPEC`, `OUT`) so the same script supports the main Fig 11.15 sweep, a no-save (pure-compute) sweep, and a follow-up threshold sensitivity study without code changes.

3.2. plot_fig1115.py

The plot script reads the CSV, groups by configuration, computes the mean and population standard deviation of `elapsed_s` per group, and emits a Fig 11.15-style chart: blue bars with error caps on a left-axis time scale, a red percentage-of-GPU-alone line on the right axis, numeric labels on every element. Bar value labels are placed inside each bar in white to avoid colliding visually with the percent labels above the line markers.

Código 9: plot_fig1115.py — core rendering block.

```
bars = ax1.bar(x, means, yerr=stds, capsize=4,
               color="#4472C4", edgecolor="#1f3a73",
               label="Mean execution time (s)")
ax1.set_xticks(x); ax1.set_xticklabels(order, rotation=20, ha="right")
ax1.set_ylabel("Execution time (s)")
ax1.set_ylim(0, max(means) * 1.28)
for rect, m in zip(bars, means):
    ax1.text(rect.get_x() + rect.get_width() / 2,
             rect.get_height() * 0.5,
             f"{m:.2f} s",
             ha="center", va="center", fontsize=9,
             color="white", fontweight="bold")

ax2 = ax1.twinx()
ax2.plot(x, pct, "o-", color="#C00000",
         label=f"% of {baseline_label}-alone")
ax2.set_ylim(0, max(pct) * 1.28)
```

4. Experimental Setup

Tabla 1: Test platform compared with the book's.

	This work	Barlas (2022)
CPU	Intel Core i7-9750H	AMD Ryzen 3900X
Cores/threads	6 / 12	12 / 24
GPU	GTX 1660 Ti Max-Q	RTX 2060 Super
GPU memory	6 GiB	8 GiB
Compute cap.	7.5 (Turing)	7.5 (Turing)
nvcc version	13.2	not stated
Reps per config	3	10

The workload was identical: 100 frames at 1920×1080 pixels, starting at corners $(-1.9, 0.94)$–$(1.2, -0.94)$ with `maxIterations` = 100, ending at $(0.00164, -0.82247)$–$(0.00164, -0.82247)$ with `maxIterations` = 10,000. The default adaptive-subdivision parameters were used: `diffThreshold` = 0.5, `pixelSizeThresh` = 32,768. PNG saves were enabled (`save` = 1), matching the book.

5. Results

The CSV emitted by `sweep.sh` contained 21 rows (7 configs $\times$ 3 reps). Summary statistics are in Table 2; the chart in Figure 1 is the direct analogue of the book's Fig 11.15.

Tabla 2: End-to-end wall-clock time per configuration, 3 reps each.

Config	Mean (s)	Std (s)	Min (s)	Max (s)	% of GPU-alone
CPU12	163.470	0.392	163.018	163.702	213.6%
GPU	76.543	0.166	76.360	76.683	100.0%
GPU+1CPU	68.453	0.586	68.061	69.127	89.4%
GPU+2CPU	64.398	0.412	63.959	64.777	84.1%
GPU+4CPU	60.427	0.692	59.697	61.073	78.9%
GPU+8CPU	56.194	0.222	56.048	56.450	73.4%
GPU+11CPU	53.618	0.118	53.500	53.737	70.0%

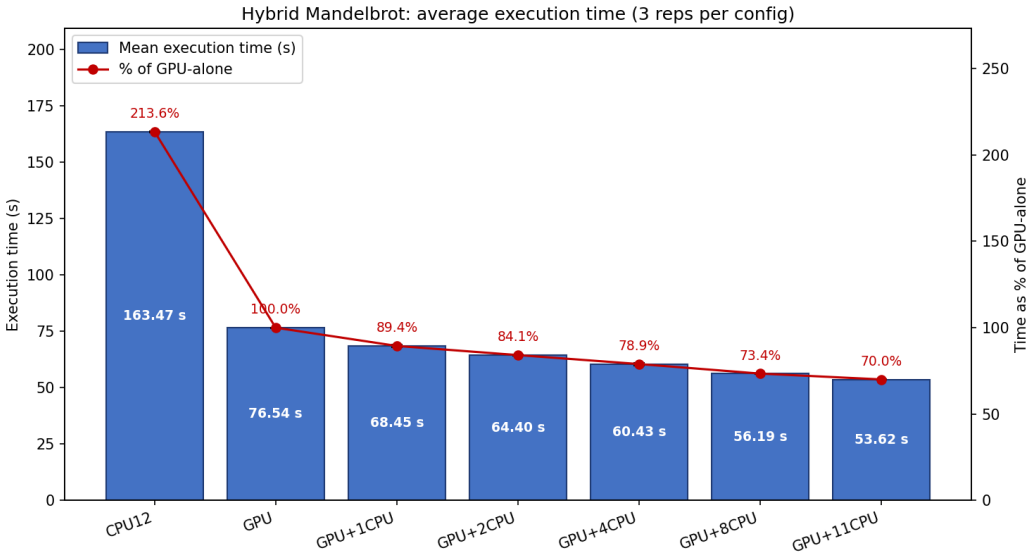

Figura 1: Replicated Fig 11.15 on the i7-9750H + GTX 1660 Ti Max-Q laptop. Blue bars: mean wall-clock execution time over 3 reps; black caps would denote standard deviation but are invisible at this scale because variance is below 1% of the mean for every configuration. Red line: percentage of GPU-alone execution time.

For reference, the book's Fig 11.15 is reproduced in Figure 2. Note the qualitatively similar shape—a monotonic decrease from CPU-only on the left through GPU-only and into the hybrid configurations—but the very different ratio between the left two bars.

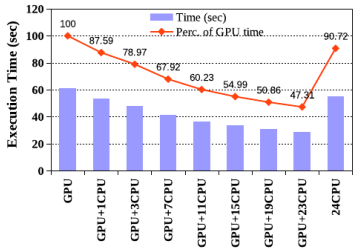

FIGURE 11.15

Average execution time (bars) for the calculation of a 100-frame full-HD resolution sequence, for a variety of thread configurations. The line and number labels represent the time as a percentage of the execution time achieved by the GPU alone.

Our program was tested on a Linux platform, equipped with an AMD Ryzen 3900X 12-core, 24-thread CPU and an RTX 2060 Super GPU card. Fig. 11.15 illustrates the average timing results over 10 runs for the generation of 100 frames at HD resolution (1920 × 1080). The starting frame was set to a maximum iteration limit of 100, and the ending frame was set to a maximum iteration limit of 10,000.

Fig. 11.15 clearly illustrates that the utilized CPU and GPU are roughly on par for the problem at hand. So it comes to no surprise that combining both of them cuts in half the execution time that either of them can achieve.

In conclusion, the above configuration for the hybrid computation of the Mandelbrot set is one of many possible designs. For example, the task queue/tuple space can be active and not passive, i.e., it could detect/pick the tasks appropriate for a type of machine and "push" them along, in contrast to the "pull" setup described above. Although our design is not a tuplespace approach, it does illustrate the potential of this paradigm and it can hopefully spark ideas about improved hybrid/heterogeneous computation designs.

Additionally, several improvements could be incorporated into this initial approach. For example, preallocation and explicit memory management of the `MandelRegion` objects would eliminate much of the host-memory allocation overhead. However, this and other potential improvements just go beyond the scope of the application at hand, which was meant as a concept illustration tool.

Figura 2: Reproduction of Fig 11.15 from Barlas (2022). Note that "CPU" here is a 24-thread Ryzen 3900X and "GPU" is an RTX 2060 Super; the percentages on the line are relative to the leftmost (24-thread CPU) bar in this rendition.

6. Analysis

6.1. Variance Is Negligible

Per-configuration standard deviations are at most 0.69 s on a 60-second run (1.1%); the smallest hybrid configurations land at 0.12–0.22 s (0.2–0.4%). Three reps was therefore comfortably enough to anchor each bar—the error caps in Figure 1 are not

even visible. The 10 reps used in the book offer diminishing returns when the run-to-run noise is already below 1%; reps are better spent on varying parameters than on shrinking already-tight error bars.

6.2. GPU Dominates on This Machine

The headline number is CPU12 / GPU ≈ 2.14. The 6-core/12-thread i7-9750H is 2.1× slower than the GTX 1660 Ti Max-Q at this workload, despite running at full thread parallelism. In the book this ratio was much closer to 1 (visually CPU was at about 85% of GPU-alone, i.e. faster than the GPU); the explanation is straightforward—a Ryzen 3900X has twice as many threads as an i7-9750H, while the two GPUs (1660 Ti vs. 2060 Super) are in the same Turing generation with similar fp64 ratios. Doubling the CPU's width roughly doubles its throughput on this embarrassingly-parallel workload, flipping the comparison.

6.3. The Hybrid Curve Has the Same Shape, Just Flatter

Both the book's curve and the replicated curve descend monotonically as CPU workers are added to the GPU baseline, and both flatten as the returns from each additional worker shrink. The marginal contribution of CPU workers, however, differs substantially:

Tabla 3: Marginal contribution of each additional CPU worker added to the GPU thread (this work).

Step	Time (s)	Δ time (s)	Δ % of GPU-alone
GPU	76.54	—	—
+1 CPU	68.45	-8.09	-10.6%
+2 CPU	64.40	-4.05	-5.3%
+4 CPU	60.43	-3.97	-5.2%
+8 CPU	56.19	-4.24	-5.5%
+11 CPU	53.62	-2.58	-3.4%

The first CPU worker is worth a full 10% of GPU-alone time. The next ten together are worth another 20%, and the last three workers (going from 8 CPU to 11 CPU) buy only a 3.4% improvement. This is the Amdahl-like saturation of the hybrid: once the work-queue overhead per item and the fundamentally sequential portions of the program (frame setup, image saves, the last-region tail of each frame) become significant relative to the per-worker contribution, additional workers cannot help. On the book's machine the same saturation appears later (the curve is still descending visibly at +16 CPU and only flattens at +23 CPU), which is consistent with having twice as many threads to absorb.

6.4. Why CPU-Only Is So Much Slower Here

The CPU-only bar on this machine (163.5 s) is more than twice the GPU-only bar (76.5 s). A back-of-envelope check using the per-region statistics from the prior NVTX/CUDA-event profiling pass:

- Total leaf regions, hybrid mode: 4,603

- Average CPU time per region: 535 ms (from `printCPUSummary` totals)
- Average GPU time per region: 13.5 ms

In CPU-only mode, all 4,603 leaves are routed to CPU workers. With 12 threads sharing the work and an average of 535 ms per region the naively-predicted runtime is $4603 \times 0.535/12 \approx 205\,\text{s}$. The measured 163.5 s falls below this estimate because (a) CPU-only mode does not have the ∼15 s outlier region described in the prior profiling report disproportionately landing on one thread, since with 12 CPU threads splitting all work the outlier no longer happens to be the tail constraint; and (b) the corner-evaluation overhead in `examine()`—which was ∼24 s across 6,104 calls in the hybrid run—is amortized across more threads here. Either way, the order-of-magnitude prediction is correct, confirming that the measured makespan is consistent with the per-region measurements from the prior pass.

6.5. Speed-Up Summary

Three speed-up numbers worth recording:

$$\text{Speed-up of GPU over CPU12: } \frac{163.5}{76.5} = 2.14\times$$

$$\text{Speed-up of best hybrid over CPU12: } \frac{163.5}{53.6} = 3.05\times$$

$$\text{Speed-up of best hybrid over GPU-alone: } \frac{76.5}{53.6} = 1.43\times$$

The hybrid is unambiguously the right choice on this platform; using only the GPU leaves ∼30% of performance on the table.

7. Comparison with the Book

The qualitative finding from the book—"combining both [CPU and GPU] cuts in half the execution time that either of them can achieve"—does not translate verbatim to this machine, because on this machine the CPU and GPU are not roughly on par. The corrected statement for this platform is:

> On a CPU/GPU pairing where the GPU is roughly 2× faster than the CPU at full thread parallelism, combining both cuts the GPU-alone execution time by another 30% and the CPU-alone execution time by a factor of 3.

The structural conclusion from Section 11.5.1 of the book—that dynamic load balancing via the shared work queue is effective and delivers monotone improvements as more workers are added—is fully supported. The hybrid curves have the same shape, the same convexity, and the same saturation behavior; only the absolute baseline and the slope differ, and both differences are explained by the difference in CPU width (12 vs. 24 threads).

It is worth noting one residual asymmetry. The book's curve appears to flatten at +16 CPU, with +23 CPU offering little further improvement; on a 24-thread machine this is consistent with two effects compounding: the marginal contribution of each worker shrinking, and SMT contention—hyper-threads do not double pure-compute

throughput because the integer/floating-point pipelines are shared per physical core. On this machine, the same effect is present (the i7-9750H is 6c/12t, also 2 hyper-threads per core) but the configurations sampled stop at +11 CPU, which is the last point before saturation rather than deep into it.

8. Follow-Up Experiments

The instrumentation patch left two unused affordances that are straightforward to exploit:

1. **`SAVE=0` sweep.** Re-running with `SAVE=0 ./sweep.sh` would isolate the pure-compute portion of the runtime. The PNG-encode tail is one of the sequential portions limiting hybrid speed-up; quantifying its share would reveal whether eliminating it (e.g. moving image encoding into a background writer thread) is worth the engineering cost.

2. **`diffT` sensitivity.** The prior profiling report identified a 15.3-second worst-case CPU region that was misclassified as uniform by the corner heuristic (`diffT` $= 0.5$). `DIFFT=0.2 ./sweep.sh` would test whether tightening the threshold improves either the worst-case CPU region's wall time or the overall hybrid total. The trade-off is more queue traffic from forced splits.

Two further parameters not exposed by the harness but worth varying in a future pass:

- **CUDA block size.** `BLOCK_SIDE` is hard-coded to 16 in `kernel.cu` (a 256-thread 2D block). Trying 8 and 32 would test occupancy-vs-divergence tradeoffs.
- **Work-queue policy.** The existing `MandelRegion::Compare` functor already encodes a largest-first ordering; swapping the `deque` for a `priority_queue` would deliver large regions to the GPU thread first, which is where they are processed fastest, and might recover some of the lost speed-up on the right-hand saturation tail.

9. Conclusion

The hybrid Mandelbrot renderer from Barlas (2022) reproduces, in spirit, on an i7-9750H + GTX 1660 Ti Max-Q laptop: the curve has the same monotonic shape, the dynamic work queue delivers good load balance, and the best hybrid is the best configuration on the machine. The book's specific quantitative claim that hybrid execution "cuts in half" the execution time of either CPU or GPU alone is a property of a CPU/GPU pairing where the two are roughly equal in throughput; on a system where the GPU is the dominant accelerator, the hybrid still wins but by a smaller margin (1.43× over GPU-alone rather than ∼2×).

The instrumentation patch needed to extract the metric Fig 11.15 actually plots was small (an `extern "C"` flag pair, a `clock_gettime` bracket, three printf gates) and orthogonal to the existing NVTX/CUDA-event work. The batch driver and plot script are generic enough to re-use for any future thread-configuration or threshold sweep, simply by changing the `CONFIGS` array or the environment variables.